Exercise 9 — AI Solution Verification Challenge

Exercise Description

Take an AI-provided solution containing a subtle bug (a merge sort), then apply **all three verification strategies** — Collaborative Solution Verification, Learning Through Alternative Approaches, and Developing a Critical Eye — document the verification process and what you learned, and implement the final verified solution.

Files: - `merge_sort.py` — `merge_sort_buggy` (the flawed AI version, kept for reference) and `merge_sort` (the verified fix). - `test_merge_sort.py` — tests that verify the fix and *demonstrate the bug*.

The AI-provided solution and its bug

The sample merge sort looks correct but the "copy remaining left elements" loop advances the **wrong index**:

```
while i < len(left):
    result.append(left[i])
    j += 1          # BUG: should be i += 1
```

Because `i` never changes, `i < len(left)` stays true forever. This is **not** just a wrong result — it's an **infinite loop** that triggers whenever the left half still has elements after the right half is exhausted (which is common).

Strategy 1 — Collaborative Solution Verification (trace a concrete example)

Merging `left = [1, 4]`, `right = [2, 3]`: 1. `1 < 2` → append 1, `i=1` 2. `4 < 2`? no → append 2, `j=1` 3. `4 < 3`? no → append 3, `j=2` → right exhausted, main loop ends with `i=1, j=2` 4. Leftover-left loop: `i(1) < len(left)(2)` true → append `left[1]=4`, `j=3` … `i` still 1 → **loops forever appending 4**.

Tracing by hand immediately exposed that the terminating index is wrong.

Strategy 2 — Learning Through Alternative Approaches

I compared two correct ways to drain the remainder: - **Index loops done right:** `while i < len(left): result.append(left[i]); i += 1`. - **Slice extend (cleaner, less error-prone):** `result.extend(left[i:]); result.extend(right[j:])`.

The slice form removes the class of "advance the wrong counter" bug entirely, so I chose it for the verified version. Considering alternatives made the fix both correct *and* more robust.

Strategy 3 — Developing a Critical Eye (adversarial tests)

I wrote tests targeting exactly the risky cases: empty list, single element, already-sorted, reverse-sorted, duplicates, negatives, and 200 random arrays checked against Python's `sorted`. I also added a **non-termination test** that runs the buggy version in a daemon thread and asserts it does *not* finish within 1 second — turning "it hangs" into an automated check.

Final verified solution

`merge_sort` returns a new sorted list, does not mutate input, uses `<=` in the merge to stay **stable**, and drains remainders with slice-extend. Test run:

```
test_buggy_infinite_loops_on_leftover_left ... ok
test_fixed_does_not_mutate_input ... ok
test_fixed_matches_sorted_on_random_data ... ok
test_fixed_sorts_correctly ... ok
Ran 4 tests in 1.029s — OK
```

Reflection Questions

1. **How did your confidence change after verification?** It flipped from "looks fine" to "provably correct" — hand-tracing found the bug in seconds, and the random/property test gives ongoing confidence.
2. **What needed the most scrutiny?** The remainder-copy loops — the exact spot the bug lived. The main merge loop looked fine and was fine.
3. **Which technique was most valuable?** Hand-tracing a concrete example (Strategy 1) found the bug fastest; property-testing against `sorted` (Strategy 3) gives durable protection against regressions.

Submission for Exercise 9 — AI Solution Verification Challenge

1. Scenario & AI solution: a merge sort whose leftover-left loop advances `j` instead of `i` — see `merge_sort_buggy` in `merge_sort.py`.

2. Verification process (all three strategies): collaborative trace of `[1,4]` + `[2,3]` (found the infinite loop); alternative approaches (index-loop vs slice-extend, choosing the

more robust slice form); a critical-eye adversarial test suite including empty/single/sorted/reverse/duplicate/negative/random cases and a non-termination check.

3. What I learned: the bug is an infinite loop, not a wrong value; considering alternatives can eliminate a whole bug class; and "it hangs" can be made into an automated test with a thread timeout.

4. Final verified solution: `merge_sort` in `merge_sort.py` — stable, non-mutating, O(n log n), all tests passing (`Ran 4 tests — OK`).

5. Reflection answers: provided above.